MLDS 490 - Assignment 2: Deep Q-Network

Zikun Zheng | GitHub: ZikunZheng1129 | Repository: mlds490-hw2

https://github.com/ZikunZheng1129/mlds490-hw2

Method

Both tasks use standard DQN: a neural Q function, uniform replay, a target network, and epsilon-greedy exploration. The target is **y = r + gamma (1 - terminated) max Q_target(next state, action)**. Adam minimizes mean Huber loss against detached targets; gradient norm is clipped at 10. Epsilon decreases linearly from 1.

Parameter	CartPole-v1	MsPacman-v5
Discount factor	0.95	0.99
Adam learning rate	0.0005	0.0001
Batch size	64	32
Replay capacity	50000	50000
Warm-up decisions	1000	10000
Decisions per update	1	4
Updates per target copy	250	2000
Epsilon decay decisions	30000	500000
Final training epsilon	0.02	0.05
Training seed	7	7

Networks. CartPole: 4 inputs, two 128-unit ReLU layers, 2 Q outputs. MsPacman: four 88 x 80 frames; convolution layers 32 (8 x 8, stride 4), 64 (4 x 4, stride 2), and 64 (3 x 3, stride 1); 256 ReLU units; 9 Q outputs. Outputs are raw action values, with no softmax.

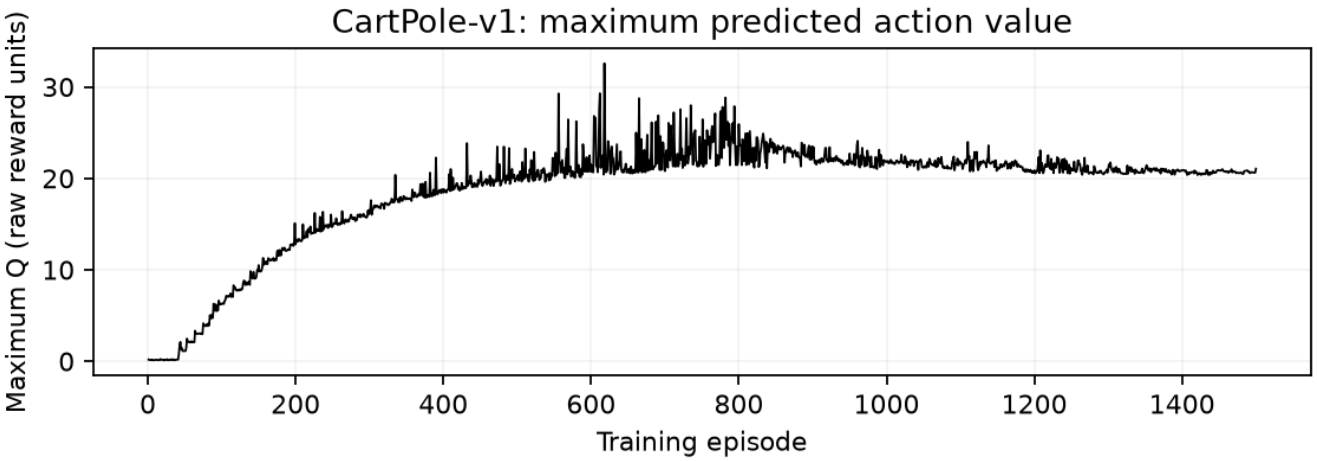
MsPacman preprocessing. The assignment's crop, grayscale and contrast mask are used. Replay stores uint8 images; the network computes (pixel - 128)/128. Four frames provide motion information. Training rewards are sign-clipped; all reported scores are raw game points. ALE/MsPacman-v5 uses frame skip 4, sticky-action probability 0.25 and 9 actions. Life loss does not end a full-game episode.

Evaluation. The highest greedy development mean selects the saved model; ties keep the earlier model. Development seeds start at 40000. Each selected model is evaluated without updates or exploration on 500 complete episodes with independent seeds. These scores never select the model. Standard deviations below use ddof=1.

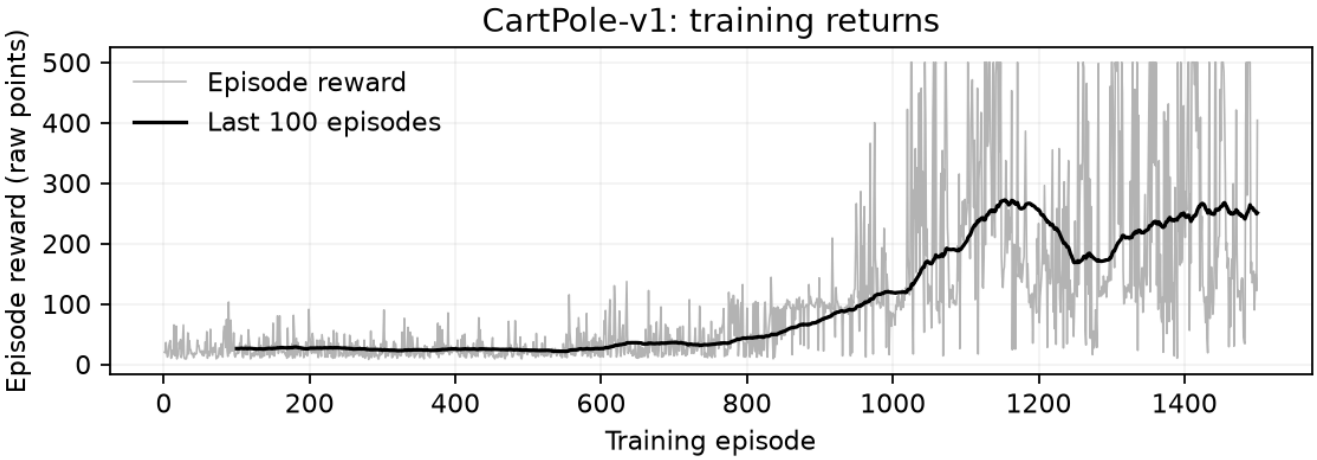
Validation. Random, untrained and selected policies share held-out seeds starting at 80000. Untrained and selected policies are greedy; the random policy samples uniformly. Paired 95% intervals use 2,000 bootstrap resamples. These compare learned behavior beyond software tests. Different actions can produce different trajectories even with matched seeds. One training seed per task limits claims about retraining.

Shared edge cases. Both termination and truncation end rollouts, but only true termination removes the Bellman bootstrap. Learning waits for warm-up and a full batch. Replay is bounded and copies observations. Non-finite losses fail visibly. Tests cover these branches, target-network freezing and unchanged weights during evaluation.

CartPole: training behavior



Maximum Q. Each point is the maximum over all visited states and all action outputs in that training episode. The online network changes during the episode. Q values are predictions, not measured game scores; a larger Q alone does not prove learning.

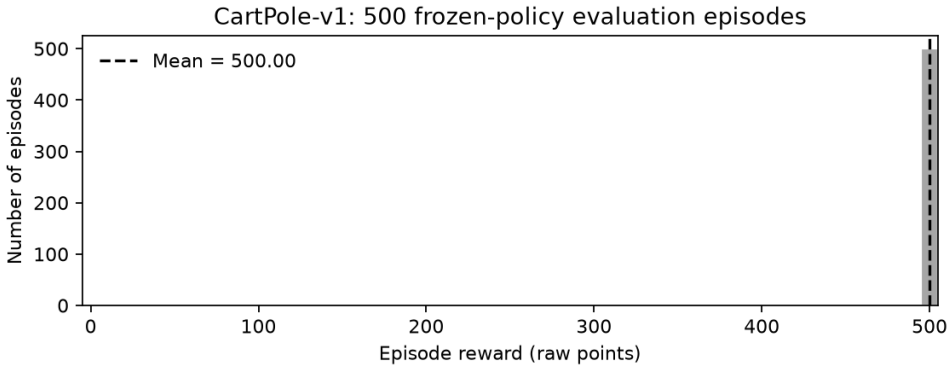


The first and final 100 training episodes average **25.98** and **250.68** raw reward, respectively (difference +224.70). The 100-episode moving average begins only when 100 complete episodes exist. Exploration changes over training, so the fixed-policy held-out comparisons on the next page provide a separate check of actual learned behavior.

Frozen evaluation uses the development-selected checkpoint from episode **1300**. The training curve includes exploratory actions and subsequent parameter updates; its final moving average therefore need not match the selected checkpoint's greedy evaluation score.

The largest predicted Q is **32.58**. With reward 1 per step and gamma=0.95, discounted return cannot exceed **20.00**. The excess indicates Q-value overestimation or function-approximation error; Q growth is not a correctness certificate.

CartPole: evaluation and validation



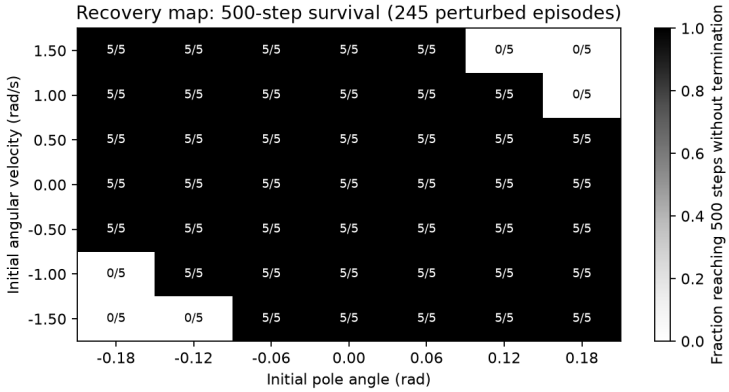
500 episodes: mean 500.00, sample SD 0.00, range 500-500. 0 true terminations; 500 time truncations. Evaluation seeds: 100007-100506.

50-seed validation policy	Mean reward	Sample SD
Random	22.46	12.29
Untrained	9.34	0.66
Selected	500.00	0.00

Selected minus random: 477.54; 95% paired interval [473.82, 480.68].

Selected minus untrained: 490.66; 95% paired interval [490.48, 490.84].

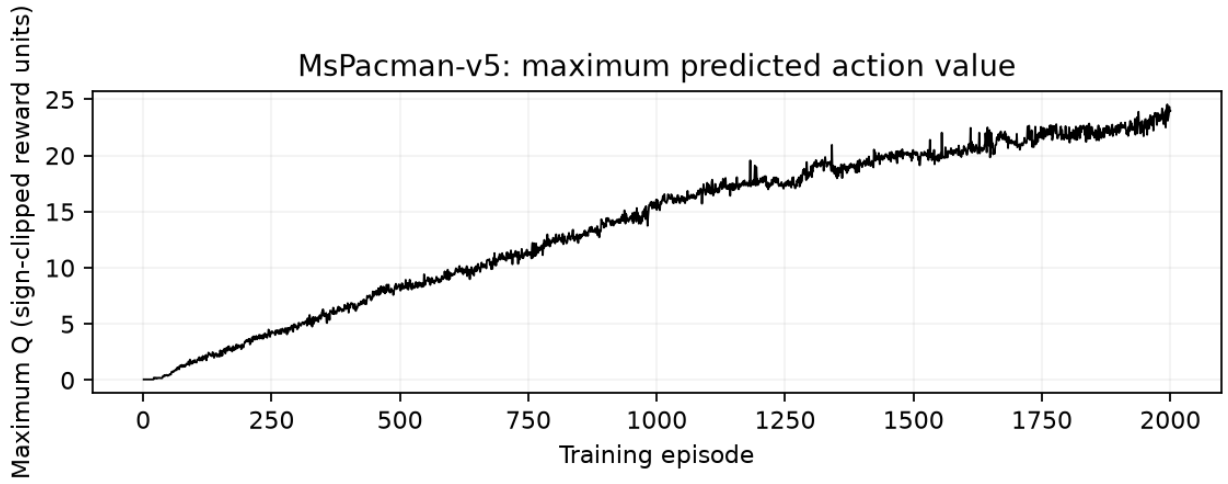
The paired interval supports a positive improvement under this evaluation protocol.



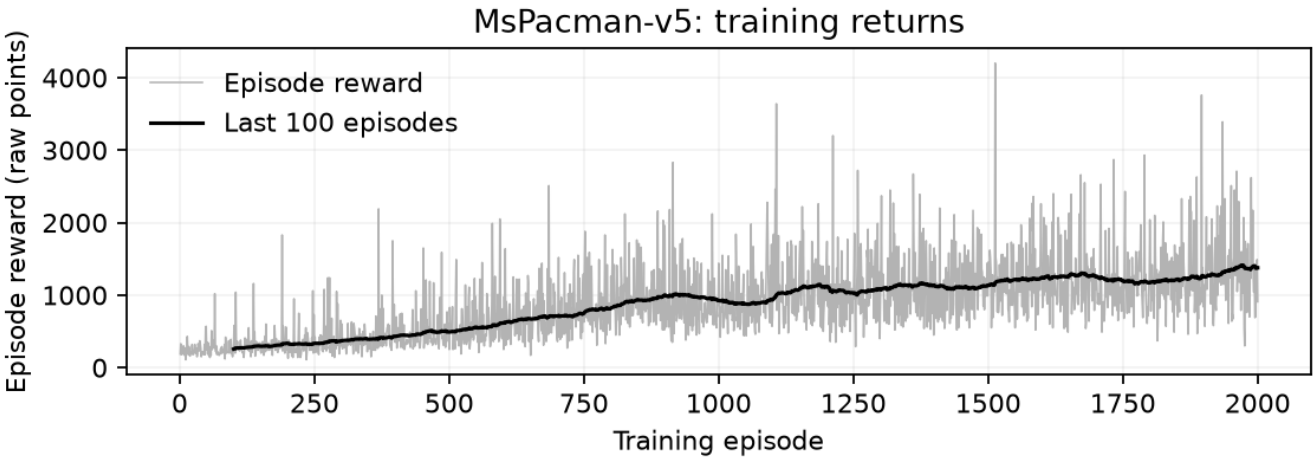
Objective and edge cases. 500/500 standard episodes reached 500 steps. The nonstandard recovery map tests starting angles and angular velocities beyond normal resets, with small independent jitters. Each cell shows successes/trials. Across 245 trials, 87.8% reached 500 steps; mean length was 439.1. Difficult initial states reveal a recovery limit. Time-limit endings keep the bootstrap term; physical failure removes it.

Ten-step target audit: 10 recorded transitions, all match: True. Terminal: not_observed; truncation: not_observed. Unobserved cases are not passes. The full states/image summaries, Q values, actions and rewards are retained in validation_first10_steps.json.

MsPacman: training behavior



Maximum Q. Each point is the maximum over all visited states and all action outputs in that training episode. The online network changes during the episode. Q values are predictions, not measured game scores; a larger Q alone does not prove learning.

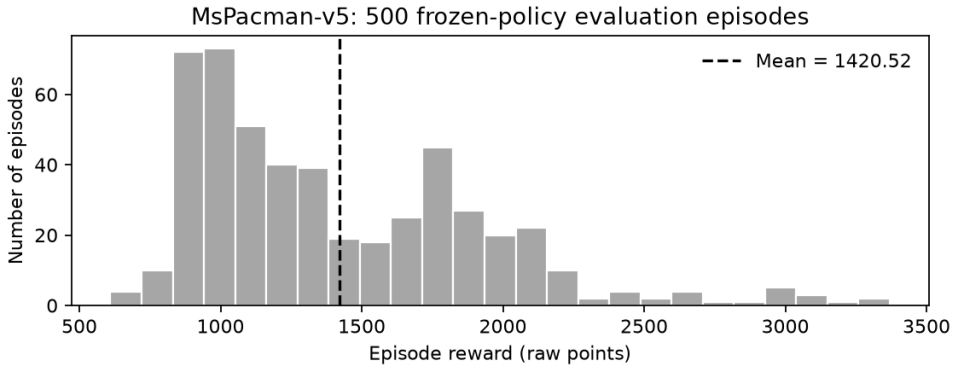


The first and final 100 training episodes average **257.00** and **1377.70** raw reward, respectively (difference +1120.70). The 100-episode moving average begins only when 100 complete episodes exist. Exploration changes over training, so the fixed-policy held-out comparisons on the next page provide a separate check of actual learned behavior.

Frozen evaluation uses the development-selected checkpoint from episode **1500**. The training curve includes exploratory actions and subsequent parameter updates; its final moving average therefore need not match the selected checkpoint's greedy evaluation score.

The last 5 development means range from 955.0 to 1583.0 points, ending at 1267.0. Late development scores rise and fall; the run does not demonstrate stable or monotonic convergence. Development scores select a checkpoint; the independent 500-episode result on the next page is the final performance estimate.

MsPacman: evaluation and validation



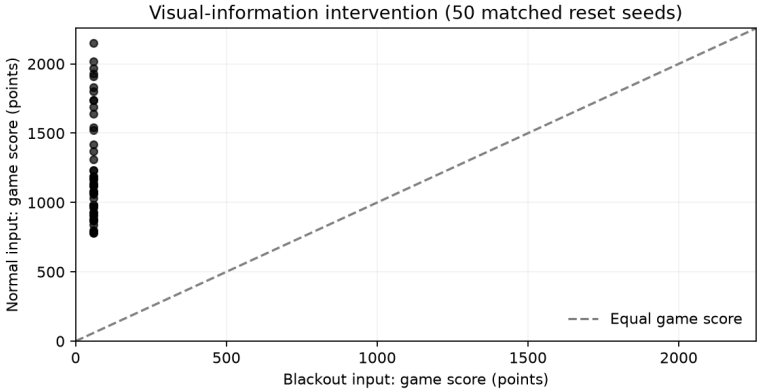
500 episodes: mean 1420.52, sample SD 511.69, range 610-3370. 500 true terminations; 0 time truncations. Evaluation seeds: 100007-100506.

50-seed validation policy	Mean reward	Sample SD
Random	279.80	147.93
Untrained	60.00	0.00
Selected	1234.00	382.45

Selected minus random: 954.20; 95% paired interval [841.00, 1084.63].

Selected minus untrained: 1174.00; 95% paired interval [1073.76, 1287.80].

The paired interval supports a positive improvement under this evaluation protocol.



Visual-input edge case. The nonstandard plot compares normal observations with all-zero images on matched seeds; points above the diagonal favor normal vision. Blackout mean: 60.00. Normal minus blackout: 1174.00, 95% interval [1073.76, 1287.80]. Only network inputs change; full-game endings and raw scoring remain intact. Blackout also introduces distribution shift, so it is not a pure test of perception.

Ten-step target audit: 10 recorded transitions, all match: True. Terminal: not_observed; truncation: not_observed. Unobserved cases are not passes. The full states/image summaries, Q values, actions and rewards are retained in validation_first10_steps.json.

Running the code and validation

Both runs used one NVIDIA GeForce RTX 2080 Ti on mlds-deepdish3, visible GPU 2. Use Python 3.12 and run from the repository root. Select one available GPU on deepdish. The fresh output folders below preserve the supplied experiment records.

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install torch==2.14.1 --index-url https://download.pytorch.org/whl/cu126
python -m pip install -r requirements.txt
export CUDA_VISIBLE_DEVICES=2
python train.py --task cartpole --episodes 1500 --seed 7 --learning-rate 0.0005 \
  --eval-every 50 --eval-episodes 20 --epsilon-decay-steps 30000 \
  --device cuda:0 --output results/rerun/cartpole
python train.py --task mspacman --episodes 2000 --seed 7 --learning-rate 0.0001 \
  --eval-every 100 --eval-episodes 10 --epsilon-decay-steps 500000 \
  --device cuda:0 --output results/rerun/mspacman
python validate.py --task cartpole --run results/rerun/cartpole --episodes 50 \
  --grid-size 7 --grid-repeats 5 --device cuda:0
python validate.py --task mspacman --run results/rerun/mspacman --episodes 50 \
  --device cuda:0
python -m pytest --cov=dqn --cov=train --cov=validate --cov=build_report
python build_report.py --results results/rerun --output hw2_report_rerun.pdf \
  --repo-url https://github.com/ZikunZheng1129/mlds490-hw2
```

Training automatically performs the 500-episode evaluation. Validation compares random/untrained/selected policies, runs the CartPole recovery grid or MsPacman blackout experiment, and saves the ten-step target audit. To validate the supplied models, replace results/rerun/<task> with results/<task>. The JSON/CSV files contain configurations, source hashes, raw outcomes and intervals. Checkpoints omit replay and random-generator state; reproduction starts a fresh run.

Given an additional three months

Month 1: Repeat each setup with at least 10 training seeds and equal interaction budgets. Report confidence intervals across training runs, not only evaluation episodes. Keep development and final test seeds separate.

Month 2: Change one factor at a time: remove replay, vary target-copy intervals, compare Huber and squared loss, and vary exploration. For MsPacman, compare one versus four frames and clipped versus raw training rewards. Compare score, learning speed and TD error at equal budgets.

Month 3: Expand CartPole's initial-state grid and test changes in mass, pole length and observation noise. Test MsPacman occlusion, delayed frames and sticky-action rates. Inspect failures and confirm improvements on untouched test seeds; retain standard environments as the main benchmark.

References

MLDS 490 Assignment 2, Week 6 DQN lecture and Lab Week 6. PyTorch DQN tutorial; ALE environments; Gymnasium time limits.